Validating distillation performance by Monte Carlo: a guide to `qmsd.mc_distill` and the next steps

prime_msd project notes for Aanha

28 October 2026

Contents

1 Where this fits **1**
- 1.1 The paper . . . 1
- 1.2 Your two steps . . . 2
- 1.3 Reading this document . . . 2

2 Background in one page **2**

3 The model the engine computes **3**

4 The codes and their reference numbers **4**

5 Inside `qmsd/mc_distill.py` **6**

6 Exercises **8**

7 Toward the circuit simulation **11**

8 Deliverables and checklist **12**

A Getting G_0 and H_1 **12**

B The exact curve in three lines **13**

C Files **13**

1 Where this fits

1.1 The paper

The paper we are assembling has three parts.

1. **New codes.** Magic-state distillation codes with overhead exponent $\gamma < 1$ at $p = 7$ (the first ever at that dimension), at $p = 17$ and 19, and the Reed–Solomon ladder from $p = 23$ to 97; plus your catalog entries at $p = 3, 5, 7$ and the first cataloged codes at $p = 11, 13$.
2. **A structural result.** No small $\gamma < 1$ code exists for $p \leq 13$ in the Reed–Muller framework; the weighted-support construction that succeeds at $p = 7$ is the one way around it, and it does not help at $3, 11, 13$.

3. **An exact performance engine.** Every code in the catalog gets its output-error curve, threshold and distillation cost computed exactly rather than by the leading-order estimate that papers usually quote.

Every cost and threshold number in part 3 rests on one piece of code, `qmsd/performance.py`. It reproduces a published table to every printed digit, which is reassuring, but nobody has checked it against an *independent* method, and nobody anywhere has published a noisy-circuit simulation of any qudit triorthogonal code. Those two checks are your part of the paper.

1.2 Your two steps

1. **Channel-level Monte Carlo** (this document). Sample the noise model the engine assumes, apply the acceptance rule exactly as the engine defines it, and show that the sampled output error and acceptance probability land on the engine's exact curves. Deliverable: Figure 1 with your own points on it, for at least four codes, and a short table.
2. **Circuit-level simulation in `sdim`.** Replace the idealised channel by a noisy encoding-and-measurement circuit for one or two small codes and measure how far the real output error departs from the channel-level curve. Deliverable: a notebook and a plot of simulated against predicted performance.

Step 1 is deliberately easy: the sampler is 44 lines, the exact curves are already computed, and the codes have at most 44 qudits. Its value is that it validates everything downstream, and that every piece of it (the noise model, the acceptance test, the two error notions, the statistics) reappears unchanged in step 2. If you do step 1 carefully, step 2 is mostly circuit construction.

1.3 Reading this document

Section 2 is background you may already know; skim it and come back for the vocabulary boxes. Section 3 defines exactly what is being computed. Section 4 gives the three codes and the numbers you must reproduce. Section 5 walks through the sampler. Section 6 has ten exercises in order of difficulty; each is preceded by a paragraph saying what it teaches and followed by a box with the expected answer. Section 7 is the bridge to the circuit simulation. The appendices are copy-paste snippets.

2 Background in one page

Qudits and the field $\mathbb{F}_p$. A qudit of dimension p (a prime) has basis states $|0\rangle, \ldots, |p-1\rangle$. Its Pauli operators are $X|a\rangle = |a+1\rangle$ and $Z|a\rangle = \omega^a|a\rangle$ with $\omega = e^{2\pi i/p}$, arithmetic mod p. A Z-type error on n qudits is $Z^{j_1} \otimes \cdots \otimes Z^{j_n}$ and is described by the vector $j = (j_1, \ldots, j_n) \in \mathbb{F}_p^n$. Everything in this document is linear algebra over $\mathbb{F}_p$, the integers mod p.

Magic states and distillation. Clifford gates (the ones a stabiliser code can do transversally) are not universal. The missing ingredient is supplied by *magic states*, specific single-qudit states that, consumed by a Clifford circuit, implement a non-Clifford gate. Preparing them directly gives a noisy version. *Distillation* takes n noisy copies, runs a Clifford circuit plus measurements, and outputs k copies with a much lower error, at the cost of sometimes rejecting the whole batch. The relevant code family is *triorthogonal*: a condition on the generator matrix that guarantees the transversal non-Clifford gate acts as a logical non-Clifford gate, so that the encoded magic states come out clean.

Codes, stabilisers, syndromes. A CSS code is given by two matrices over $\mathbb{F}_p$: one whose rows are X-type stabilisers and one whose rows are Z-type stabilisers. For distillation only the Z errors matter (the magic state's noise, after *twirling*, is pure Z noise), and the stabilisers that detect them are the rows of G_0. Measuring a stabiliser row g on an error j returns the *syndrome* $g \cdot j \bmod p$. All syndromes zero means "looks clean". The *logical operators* are rows h_i that commute with the stabilisers; $h_i \cdot j \neq 0$ means output i is corrupted.

Distance and overhead. The *distance* d is the smallest number of qudits an undetected error can occupy. An error of *weight* w (number of nonzero entries of j) with $w < d$ either triggers a syndrome or is harmless; so the output error scales as ε^d. Using a code recursively, the number of raw states per final output scales as $(\log 1/\varepsilon_{\text{target}})^\gamma$ with $\gamma = \ln(n/k)/\ln d$; $\gamma < 1$ is called sublogarithmic and is the prize.

Reed–Muller codes and puncturing. The codes in our catalog are built from evaluation codes: the rows of the generator are polynomials of degree $\leq r$ in m variables evaluated at all p^m points of $\mathbb{F}_p^m$. *Puncturing* k columns (deleting k evaluation points) and *shortening* (keeping only the codewords that vanish there) produce the $[[p^m - k,\, k,\, d]]_p$ magic code; the k deleted points are the outputs. Which k points you delete is the whole search problem, and it is what your catalog entries record.

Vocabulary

$[[n, k, d]]_p$: a quantum code on n qudits of dimension p encoding k logical qudits with distance d. **Weight** $\mathrm{wt}(j)$: number of nonzero entries. **Syndrome**: the vector $G_0 j \bmod p$. **Post-selection**: keeping a run only if a condition holds (here: all syndromes zero) and discarding it otherwise. **Twirling**: averaging a noise channel over a group of gates so that it becomes diagonal (a probabilistic mixture of Paulis); here it turns arbitrary magic-state noise into independent Z^j errors. **Depolarising noise**: each qudit independently gets a random nontrivial error with probability ε. **Canonical logical row**: the representative h_i of the i-th logical operator that is δ_{ij} on the k output positions; unique modulo stabilisers.

3 The model the engine computes

A triorthogonal code is specified for our purposes by two integer matrices over $\mathbb{F}_p$: the stabiliser rows G_0 (the shortened generator; call its row span A) and the k canonical logical rows H_1 (one per output). Both come straight out of the catalog, see Appendix A. The simulation and the exact engine use *nothing else*: the code is these two matrices.

Noise. Each of the n input magic states independently suffers a Z^{j_l} error with $j_l \neq 0$ with probability ε, the nonzero value uniform on $\mathbb{F}_p^\times = \{1, \ldots, p-1\}$. This is the twirled depolarising noise of arXiv:2610.08012; the repo's older δ is $\delta = \frac{p}{p-1}\varepsilon$ (the probability of any Z^j including the identity, weighted uniformly). The whole error is a vector $j \in \mathbb{F}_p^n$ whose weight is binomially distributed with mean $n\varepsilon$.

Acceptance. The protocol measures the stabilisers and keeps the round iff every syndrome is trivial: $G_0\, j = 0$, i.e. $j \in A^\perp$. The acceptance probability $P_{\text{acc}}(\varepsilon) = \Pr[G_0 j = 0]$ is the yield of the round; $1/P_{\text{acc}}$ multiplies the cost.

Failure. An accepted round delivers a wrong output i iff the logical row sees the error: $(H_1 j)_i \neq 0$. Two error notions matter:

$$\varepsilon_{\text{out}}^{\text{avg}}(\varepsilon) = \frac{1}{k}\sum_{i=1}^{k} \Pr[(H_1 j)_i \neq 0 \mid \text{accept}] \qquad \text{(per output; what a recursion consumes)},$$
$$\varepsilon_{\text{out}}^{\text{blk}}(\varepsilon) = \Pr[H_1 j \neq 0 \mid \text{accept}] \qquad \text{(block: any output wrong; the paper's formula)}.$$

For $k = 1$ they coincide. For $k > 1$ the block error is between $1\times$ and $k\times$ the per-output error, and tends to $k\times$ as $\varepsilon \to 0$ because low-weight errors rarely hit two outputs at once. When one distillation round feeds the next, each output is used separately, so the per-output number is the physically relevant one; published tables usually quote the block number because it is easier to compute.

Leading order and threshold. For small ε, $\varepsilon_{\text{out}} \approx c\,\varepsilon^d$, where d is the code distance and $c = L_d/(p-1)^d$ with L_d the number of weight-d accepted logical errors (per output for the average notion). The *threshold* ε^* is the smallest root of $\varepsilon_{\text{out}}(\varepsilon) = \varepsilon$: above it the round makes the states worse, so a raw magic state must be better than ε^* for distillation to start.

How the engine gets exact numbers. The probability that a random error lands in the dual of a code C is, by the MacWilliams identity, $\Pr[j \in C^\perp] = |C|^{-1}\sum_{c\in C}(1-\eta)^{\text{wt}(c)}$ with $\eta = p\varepsilon/(p-1)$. So the engine only has to enumerate the *small* space A ($p^{\dim A}$ words; $\dim A = 2, 5, 3$ for the three codes below) and the cosets $A + h_i$, tally their weights (the *weight enumerator*), combine the integer tallies into one polynomial and evaluate it in `mpmath` at any ε. That is why the exact curves in Figure 1 go smoothly down to 10^{-11}. Your sampler has no such trick: it sees an event with probability 10^{-9} once per billion shots. Seeing exactly where it loses the curve is one of the exercises.

Vocabulary

Dual code $C^\perp$: all vectors orthogonal (mod p) to every row of C. Acceptance is "$j \in A^\perp$". **Coset** $A + h$: the set $\{a + h : a \in A\}$; the logical-failure test for output i is "$j \notin (A + \langle h_i\rangle)^\perp$". **Weight enumerator** $\{B_w\}$: how many vectors of a set have weight w. **MacWilliams identity**: relates the weight enumerator of a code to that of its dual; it lets the engine count the huge space $A^\perp$ by enumerating the small space A. **Leading order**: the first nonvanishing term $c\,\varepsilon^d$ of the Taylor expansion of ε_{out} in ε.

4 The codes and their reference numbers

Use these three first; they are small, exactly computable, and span $p = 3, 7, 23$ so that no bug can hide in a coincidence of one prime.

code	catalog id / builder	n	k	$\dim A$	d	c_ε	ε^*
$[[8,1,2]]_3$	RM(1, 2) punctured at column 1 (App. A)	8	1	2	2	2.000	0.2110
$[[44,5,3]]_7$	`imported-p7-n44-k5-*` (any of the ten)	44	5	5	3	1.111	0.1732
$[[18,5,4]]_{23}$	`imported-p23-n18-k5-4cc848da1772`	18	5	3	4	0.2874	0.3963

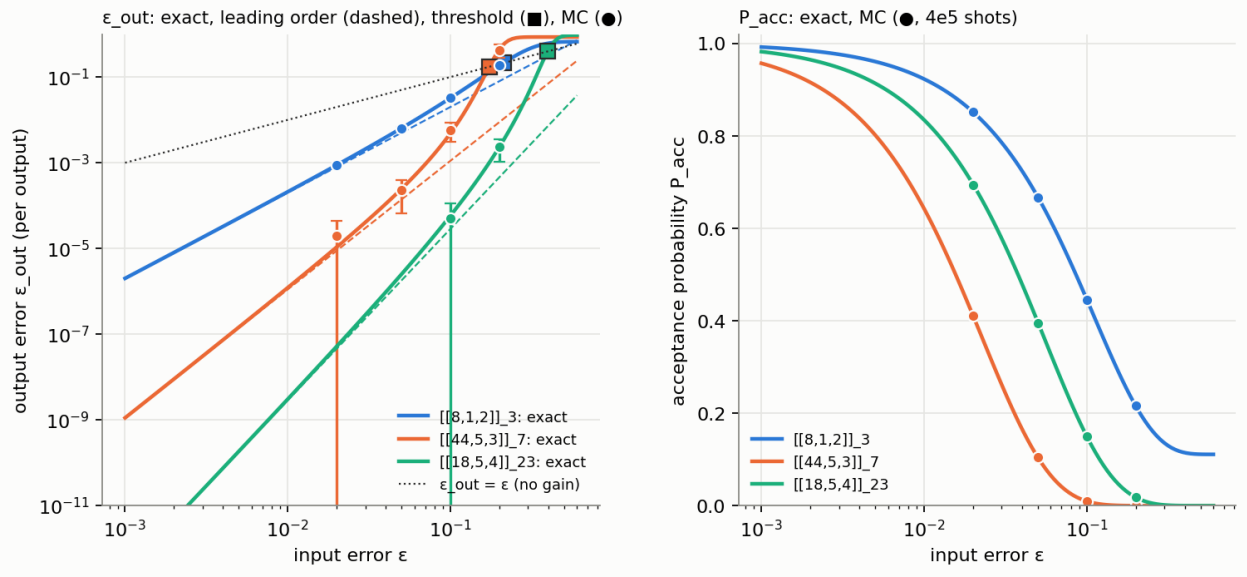


Figure 1: Left: per-output error against input error for the three codes. Solid: exact engine. Dashed: leading order $c\varepsilon^d$. Square: threshold $\varepsilon_{\text{out}} = \varepsilon$. Dots with error bars: `mc_distill.simulate` with 4×10^5 shots at $\varepsilon \in \{0.2, 0.1, 0.05, 0.02\}$. Right: acceptance probability, same conventions. Your first deliverable is this figure with your own dots on it.

A few words on each. $[[8, 1, 2]]_3$ is the smallest Reed–Muller magic code for qutrits, a $k = 1$ code with distance 2, listed in arXiv:2610.08012 Table 2; its threshold 0.211 is a published number you can check. $[[44, 5, 3]]_7$ is one of the ten equivalent $p = 7$, $m = 2$ codes in the catalog, chosen because it has $k = 5$ (so block and per-output differ) and a 5-dimensional stabiliser space (so the engine's enumeration is 7^5 words, instant). $[[18, 5, 4]]_{23}$ is the minimum-γ member of the Reed–Solomon family at $p = 23$, added to the catalog today; it has the highest distance and the smallest block of the three, and a threshold near 0.4.

Exact per-output values from the engine (these are the numbers your points must reproduce within their error bars):

		$\varepsilon = 0.2$	0.1	0.05	0.02	0.01
$[[8, 1, 2]]_3$	P_{acc}	0.2157	0.4464	0.6679	0.8515	0.9229
	ε_{out}	1.85×10^{-1}	3.24×10^{-2}	6.40×10^{-3}	8.84×10^{-4}	2.10×10^{-4}
$[[44, 5, 3]]_7$	P_{acc}	1.16×10^{-4}	9.78×10^{-3}	0.1047	0.4111	0.6426
	ε_{out}	4.42×10^{-1}	4.87×10^{-3}	2.46×10^{-4}	1.08×10^{-5}	1.22×10^{-6}
$[[18, 5, 4]]_{23}$	P_{acc}	1.81×10^{-2}	0.1501	0.3972	0.6951	0.8345
	ε_{out}	2.34×10^{-3}	5.83×10^{-5}	2.51×10^{-6}	5.23×10^{-8}	3.06×10^{-9}

Read the table once as physics. Acceptance falls as ε grows because more errors trigger a syndrome; for the 44-qudit code at $\varepsilon = 0.2$ almost every batch is rejected ($P_{\text{acc}} = 10^{-4}$) because with $n\varepsilon \approx 9$ expected errors some syndrome almost always fires. The output error falls faster than ε only below the threshold; at $\varepsilon = 0.2$ the $p = 7$ code is above its threshold of 0.17 and its output (0.44) is *worse* than its input. Between $\varepsilon = 0.02$ and 0.01 the three codes drop by factors of 4, 9 and 17, which are 2^2, 2^3 and 2^4: that is the distance showing itself.

How to read Figure 1. On a log–log plot a power law $c\varepsilon^d$ is a straight line of slope d; the dashed lines are those, and the solid exact curves peel away from them upward as ε grows, because

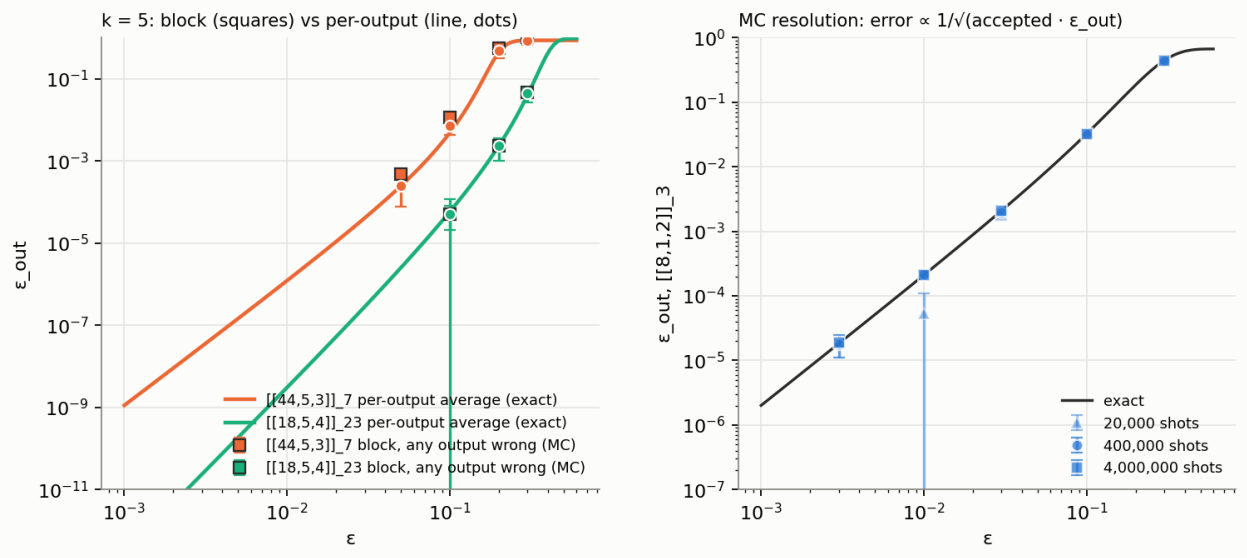


Figure 2: Left: for the two $k = 5$ codes, the block error (squares, Monte Carlo) sits above the per-output average (line exact, dots Monte Carlo) and the ratio grows toward $k = 5$ as ε falls. Right: the sampler's resolution for $[[8, 1, 2]]_3$ at 2×10^4, 4×10^5 and 4×10^6 shots; the error bar scales as $1/\sqrt{(\text{accepted shots}) \cdot \varepsilon_{\text{out}}}$, so each decade of ε_{out} costs a decade of shots.

higher-order terms (ε^{d+1}, ε^{d+2}, ...) stop being negligible. The dotted diagonal is $\varepsilon_{\text{out}} = \varepsilon$; a code only helps where its curve is below the diagonal, and the square marks where it crosses. The Monte Carlo dots sit on the exact curve where they have small error bars and become useless (huge bars, or missing entirely) once ε_{out} drops below roughly one over the number of accepted shots.

5 Inside `qmsd/mc_distill.py`

The whole sampler is one function. Read it next to Section 3; every line implements one sentence there.

```
def simulate(G0, H1, p, eps, shots, seed=0, batch=200_000) -> dict:
    G0 = np.asarray(G0, dtype=np.int64) % p # stabiliser rows, shape (dim A, n)
    H1 = np.asarray(H1, dtype=np.int64) % p # logical rows, shape (k, n)
    k, n = H1.shape
    rng = np.random.default_rng(seed)
    acc = hits = blk = 0
    done = 0
    while done < shots:
        b = min(batch, shots - done)
        # (1) the noise: each coordinate is nonzero w.p. eps, value uniform on 1..p-1
        j = np.where(rng.random((b, n)) < eps, rng.integers(1, p, size=(b, n)), 0)
        # (2) acceptance: every syndrome G0 j (mod p) must vanish
        ok = ~np.any((j @ G0.T) % p, axis=1)
        # (3) failure: for accepted shots, which outputs see the error?
        logical = ((j[ok] @ H1.T) % p) != 0 # shape (accepted, k), boolean
        acc += int(ok.sum())
        hits += int(logical.sum()) # total wrong outputs
        blk += int(logical.any(axis=1).sum()) # shots with any wrong output
```

```
        done += b
    def frac(x, m): # binomial estimate with 1-sigma error
        f = x / m if m else float("nan")
        return f, math.sqrt(f * (1 - f) / m) if m else float("nan")
    pa, pa_err = frac(acc, shots) # P_acc
    eb, eb_err = frac(blk, acc) # block error, conditioned on acceptance
    ea = hits / (k * acc) if acc else float("nan") # per-output average
    ea_err = math.sqrt(max(ea * (1 - ea), 0.0) / acc) * math.sqrt(k) if acc else float("
        nan")
    return {...}
```

Line by line.

- **(1) Sampling the noise.** `rng.random((b, n)) < eps` is a boolean matrix of which qudits are hit; `rng.integers(1, p, ...)` draws the nonzero Z power uniformly; `np.where` combines them. Each row of `j` is one error vector, i.e. one *shot*.
- **(2) Acceptance is a matrix product.** `j @ G0.T` computes every syndrome of every shot at once; `% p` reduces mod p; `np.any(..., axis=1)` asks whether any syndrome is nonzero. There is no quantum state anywhere. That is what makes the channel-level model exact and fast, and it is exactly what `sdim` will put back.
- **(3) Failure is a second matrix product.** Only accepted shots are kept (`j[ok]`); `logical[s, i]` is true if output i of shot s is wrong. `hits` counts wrong outputs (for the per-output average), `blk` counts shots with at least one (for the block error).
- **Batching.** Shots are generated 2×10^5 at a time so memory stays at a few tens of MB however large `shots` is. `seed` makes a run reproducible; change it to get an independent sample.
- **Error bars are binomial.** A count x out of m trials with true probability f has standard deviation $\sqrt{f(1-f)/m}$. P_{acc} is a fraction of `shots`; the block error is a fraction of *accepted* shots, which is why its bar is larger. The per-output average adds k correlated indicators per shot, so its bar is bounded by $\sqrt{k}$ times the block-style estimate rather than computed exactly (Exercise 6).
- **What is not modelled.** No gate noise, no measurement noise, no X errors (this code family does not post-select on them at all), no correlated noise, and no Clifford correction step. The sampler validates the engine's arithmetic under the engine's own assumptions; the circuit simulation tests the assumptions.

Vocabulary

Monte Carlo: estimating a probability by drawing random samples and counting. **Shot**: one random sample (here one error vector j on n qudits). **Accepted shot**: a shot that passed post-selection; the conditional probabilities are fractions of these, not of all shots. 1σ **error bar**: one standard deviation of the estimator; a true value should fall within 1σ about 68% of the time, within 3σ 99.7%. **Importance sampling**: drawing from a distribution that over-represents the rare events you care about and correcting each sample by a weight (true probability divided by sampling probability), so the estimate is still unbiased but far less noisy.

6 Exercises

Work in a scratch directory inside the repo, run everything with `PYTHONPATH=.` from the repo root, and keep the memory cap (`ulimit -v 4000000`) and single-thread BLAS (`OMP_NUM_THREADS=1`) on the shared boxes. Write each result into a small JSON next to your script; the plots come last. The exercises build on each other: 1–2 get you a working pipeline, 3–4 teach you what the numbers mean, 5–6 cover the two quantities the paper quotes, 7–9 improve the sampler, 10 applies it to your own codes.

Getting a point and a curve

The first exercise is a smoke test: one code, one ε, one number to compare. The only ways to fail it are a wrong matrix (Appendix A), a wrong convention for ε, or a bug in your harness, so it is worth doing before anything else.

Exercise 1: reproduce one point

Build $[[8, 1, 2]]_3$ (Appendix A) and run `simulate(G0, H1, 3, 0.05, shots=400_000, seed=1)`.

What to compare against

$P_{\text{acc}} = 0.6679$ and $\varepsilon_{\text{out}} = 6.40 \times 10^{-3}$. A run with seed 1 gave 0.6671 ± 0.0007 and $6.33 \times 10^{-3} \pm 1.5 \times 10^{-4}$. Anything within about 3σ is a pass; a 5σ miss means a bug, not bad luck.

A single point can agree by accident. A curve across two decades of ε and three primes cannot, and it also shows you the shape of the problem: the steep power-law region, the bend near threshold, and the point where sampling gives out.

Exercise 2: the whole curve

Repeat for $\varepsilon \in \{0.3, 0.2, 0.1, 0.05, 0.02, 0.01\}$ and all three codes, and plot the points over the exact curves (the exact curve is three lines of code, Appendix B). Use a log–log axis, one colour per code, error bars on every point, and a legend.

What to compare against

Figure 1. At $\varepsilon = 0.02$ and 0.01 the $[[18, 5, 4]]_{23}$ points will be zero with a zero error bar: 4×10^5 shots cannot see $\varepsilon_{\text{out}} \sim 10^{-8}$. That is correct behaviour, not a bug, and the next exercise quantifies it.

What the numbers mean

The statistical error of a counting experiment is set by the number of events counted, not by the number of trials. If N_{acc} shots are accepted and a fraction ε_{out} of them fail, you count about $N_{\text{acc}}\,\varepsilon_{\text{out}}$ failures, and the relative error of that count is $1/\sqrt{N_{\text{acc}}\,\varepsilon_{\text{out}}}$. Every decade you want to push ε_{out} down costs a decade of shots. This single fact decides when simulation is the right tool and when the exact engine is.

Exercise 3: resolution and cost

Show empirically that the relative error of ε_{out} scales as $1/\sqrt{N_{\text{acc}}\,\varepsilon_{\text{out}}}$ by running $[[8,1,2]]_3$ at $\varepsilon = 0.03$ with 2×10^4, 2×10^5, 2×10^6 shots. Then estimate how many shots a 10% measurement of $[[18,5,4]]_{23}$ at $\varepsilon = 0.01$ would need.

What to compare against

Right panel of Figure 2. For the estimate: $P_{\text{acc}} = 0.83$ and $\varepsilon_{\text{out}} = 3.1\times10^{-9}$, so 10% needs about 100 failures, i.e. 3×10^{10} accepted and 4×10^{10} shots. At $\sim10^6$ shots per second that is half a day for one point. This is why the engine exists, and why Exercise 6 matters.

The distance is usually computed algebraically (our MITM kernels find the lightest undetected error). It can also be *measured*: on a log–log plot the small-ε slope of ε_{out} is d. Doing this once makes the abstract definition concrete, and the discrepancy you will see at larger ε is the higher-order terms the leading-order formula drops.

Exercise 4: distance and leading coefficient from data

From your points at $\varepsilon \in \{0.1, 0.05, 0.02\}$ fit $\log\varepsilon_{\text{out}}$ against $\log\varepsilon$ for $[[44,5,3]]_7$ (a straight line, weighted by the error bars). The slope is the distance, the intercept the coefficient.

What to compare against

Slope ≈ 3 (exactly $d = 3$ only as $\varepsilon \to 0$; between 0.1 and 0.02 you should see 2.8 to 3.0), coefficient ≈ 1.1. The exact values are $d = 3$, $c_\varepsilon = 1.111 = L_3/6^3$ with $L_3 = 240$ weight-3 accepted logical errors per output.

The two numbers the paper quotes

Papers report a code's threshold and its block error. The threshold answers "how good must the raw magic states be for this code to help at all?"; it is a fixed point of the map $\varepsilon \mapsto \varepsilon_{\text{out}}(\varepsilon)$. Near the fixed point $\varepsilon_{\text{out}} - \varepsilon$ is small and changes sign, so a sampler needs enough shots to resolve the sign, which is harder than it sounds.

Exercise 5: threshold by bisection

Find ε^* for $[[8,1,2]]_3$ as the root of $\varepsilon_{\text{out}}(\varepsilon) - \varepsilon$, using the sampler at 10^5 shots per evaluation. Think about why bisection needs the sign of the difference and how many shots that requires near the root.

What to compare against

$\varepsilon^* = 0.2110$ (engine). The paper arXiv:2610.08012 Table 2 prints 0.211. Expect ±0.003 from the sampler.

For $k > 1$ the block error over-counts what a recursion pays for, by up to a factor k. The sampler gives both notions from the same shots, so the ratio comes for free and its ε-dependence is a direct view of how many outputs a typical undetected error touches.

Exercise 6: block against per-output

For $[[44, 5, 3]]_7$ at $\varepsilon \in \{0.3, 0.1, 0.05\}$ compute the ratio $\varepsilon_{\text{out}}^{\text{blk}}/\varepsilon_{\text{out}}^{\text{avg}}$ from the sampler's two estimates. Explain the trend.

What to compare against

Left panel of Figure 2: the ratio is about 1.3 at $\varepsilon = 0.3$ and about 2.1 at $\varepsilon = 0.05$, heading to $k = 5$. At high ε the accepted errors are heavy and hit several outputs at once; at low ε a weight-3 error hits one output.

Improving the sampler

Now modify the code. Keep the original function untouched (the test suite uses it) and work on a copy. The first improvement is honest statistics for the per-output estimate: the shipped bar is a bound because the k indicators of one shot are correlated (an error that corrupts output 1 is more likely than average to corrupt output 2 as well), and the clean way to handle correlated indicators is to treat the per-shot count as the random variable.

Exercise 7: better error bars

Modify a copy of `simulate` to accumulate, per accepted shot, the number of wrong outputs $w \in \{0, \ldots, k\}$ (its sum and its sum of squares), and compute the variance of w/k directly. Compare the new bar with the old one at $\varepsilon = 0.05$ for $[[44, 5, 3]]_7$.

What to compare against

The new bar must be smaller than or equal to the old one and never below the block-style bar divided by $\sqrt{k}$. Report both.

The engine and the sampler meet in the weight distribution: the engine computes exactly how many undetected errors there are of each weight, the sampler sees them at the rate the noise produces them. Comparing the two bin by bin is the most detailed check possible, and it is also how you would notice a subtle bug (a wrong coset, a missed stabiliser) that the totals could hide.

Exercise 8: the weight histogram and MacWilliams

Modify a copy of `simulate` to histogram the Hamming weight of every *accepted* j and of every accepted j that fails output 1. Compare the low-weight bins, divided by the number of shots, against the engine's exact counts: the number of accepted errors of weight w is B_w and the probability of one of them is $(\varepsilon/(p-1))^w(1-\varepsilon)^{n-w}$.

What to compare against

For $[[8, 1, 2]]_3$: $L_2 = c_\varepsilon(p-1)^d = 2 \cdot 4 = 8$ weight-2 accepted logical errors. Your weight-2 failing-bin frequency at $\varepsilon = 0.05$ must be $8\,(0.025)^2(0.95)^6 \approx 3.7 \times 10^{-3}$ of all shots, within error.

Exercise 3 showed that direct sampling cannot reach small ε: almost every shot is error-free or lightly hit and accepted harmlessly, and the rare weight-d failures never appear. Importance sampling fixes this by choosing the error weight yourself. If you draw weight w with probability q_w and then a uniformly random pattern of that weight, the true probability of that shot is $\binom{n}{w}(\varepsilon/(p-1))^w(1-\varepsilon)^{n-w}$ divided by the number of patterns, and the shot gets weight (true probability)/(sampling probability). Averages of weighted shots are unbiased for any q_w that gives nonzero probability to every relevant weight, and putting most of q_w on $w \in \{d, \ldots, d+2\}$ is where the failures live.

Exercise 9: importance sampling

Write a sampler that draws the error *weight* w from a distribution of your choice, then a uniformly random pattern of that weight (positions and nonzero values), and reweights each shot by the true probability $\binom{n}{w}(\varepsilon/(p-1))^w(1-\varepsilon)^{n-w}$ divided by your proposal. Use it to measure $[[18, 5, 4]]_{23}$ at $\varepsilon = 0.01$ and 0.001. Report the effective sample size as well as the estimate.

What to compare against

3.06×10^{-9} at $\varepsilon = 0.01$. At $\varepsilon = 0.001$ compute the exact value yourself with Appendix B; it is close to $c_\varepsilon\, \varepsilon^4 = 2.9 \times 10^{-13}$. With 10^6 reweighted shots concentrated on $w \in \{4, 5, 6\}$ you should get both to a few percent.

Your own codes

Everything so far used codes whose exact curve is known. Your own catalog entries are bigger. The engine can probably still enumerate $[[215, 28, 4]]_3$ ($3^{\dim A}$ with $\dim A$ in the twenties is borderline) but almost certainly not $[[689, 40, 5]]_3$. For those the sampler is not a check on the engine; it *is* the performance number, and the only cross-check is the leading-order coefficient from the A_d count the search already recorded.

Exercise 10: your own codes

Run Exercise 2 on your $[[215, 28, 4]]_3$ and $[[689, 40, 5]]_3$ entries. The exact engine may refuse the larger one (too many words to enumerate; it says so). Then the sampler is the only curve we have, and your points become the reference.

What to compare against

Where the engine runs, agreement as before. Where it does not, check your points against the leading order $A_d/(p-1)^d \cdot \varepsilon^d$ from the catalog's A_d at the two smallest ε you can resolve, and say in your write-up which regime you are in.

7 Toward the circuit simulation

The channel model assumes that the only noise is an independent Z error on each input magic state, delivered before a perfect circuit. A real round also has noisy Clifford gates in the encoder, noisy syndrome measurements, idle errors while qudits wait, and X-type errors that this family

does not post-select on. The question for `sdim` is how much each of these shifts the curve, and in which direction. Before writing any circuit, write down what you expect:

- A gate error rate p_g adds terms to ε_{out} that are *first order* in p_g and independent of d: a single faulty gate late in the circuit can corrupt an output without any syndrome firing. On a log–log plot against ε the curve therefore flattens into a floor at small ε, and the floor height is the number to report. Predict it from the gate count and p_g before you simulate it.
- Measurement errors change P_{acc} more than ε_{out}: a wrong syndrome mostly rejects good rounds (lower yield) rather than accepting bad ones.
- The channel-level curve is the $p_g \to 0$ limit. Your first `sdim` run should reproduce Exercise 1 with all circuit noise switched off; if it does not, the circuit is wrong, not the engine.

Start with $[[8, 1, 2]]_3$ (8 qutrits, two stabilisers, one logical) and then $[[18, 5, 4]]_{23}$ if `sdim` supports $p = 23$; $[[44, 5, 3]]_7$ is the realistic target. The encoding circuit comes from G_0 and H_1 by the standard CSS-encoder construction; ask before building it so we agree on the gate set.

Vocabulary

Stabiliser simulator (`sdim`): simulates Clifford circuits with Pauli noise efficiently by tracking stabilisers instead of amplitudes; exact for everything in a distillation round except the magic states themselves, which enter as inputs. **Gate error rate** p_g: probability that a gate is followed by a random Pauli on its qudits. **Floor**: the ε-independent part of the output error caused by circuit noise.

8 Deliverables and checklist

1. `runs/mc/<code>_<eps>.json` per point (shots, seed, all four estimates and errors), produced by a script committed under `scratchpad/` or `docs/`.
2. Figure 1 redrawn with your points, at least four codes, the three above plus one of yours.
3. A half-page note: which points agree, the measured scaling of Exercise 3, the fitted d and c of Exercise 4, and the importance-sampling result of Exercise 9.
4. Open a pull request against `main` from your branch. Leave the `primepower_msd/` directory untouched.

A Getting G_0 and H_1

For a catalog record:

```
from qmsd.results import load_result_catalog
from qmsd.performance import record_matrices
wanted = "imported-p23-n18-k5-4cc848da1772"
rec = next(r for r in load_result_catalog() if r.artifact_id == wanted)
G0, H1 = record_matrices(rec) # integer arrays mod p; shapes (dim A, n) and (k, n)
```

For the $k = 1$ Reed–Muller code $[[8, 1, 2]]_3$ (puncture one column of $\mathrm{RM}(1, 2)$ over $\mathbb{F}_3$; any column is equivalent):

```
from qmsd.reedmuller import r_max, rm_generator
from qmsd.triorthogonal import build_triorthogonal_code
from qmsd.performance import canonical_logical_rows
p, m = 3, 2
r = r_max(m, p); G = rm_generator(r, m, p)
```

```
G0 = build_triorthogonal_code(p, m, r, [1], G=G)["G0"]
H1 = canonical_logical_rows(G, [0], p)
```

B The exact curve in three lines

```
from qmsd.performance import profile_exact, leading_order, threshold
prof = profile_exact(G0, H1, p, kinds=("avg",)) # p**dim(A) words
print(float(prof.p_accept(0.05)), float(prof.eps_out(0.05, "avg")))
lo = leading_order(prof, "avg"); th = threshold(prof, "avg")
print(lo.d, lo.c_eps, th.eps)
```

The block kind (`kinds=("avg","block")`) costs $p^{\dim A+k}$ words and is refused above 5×10^7; for $k = 5$ at $p = 23$ that is already too many, which is why the block error in Figure 2 comes from the sampler.

C Files

- `qmsd/mc_distill.py` is the sampler and `qmsd/performance.py` the exact engine. The test suite compares them in `tests/test_performance.py` (`test_monte_carlo_agrees`).
- `PERFORMANCE.md`: the engine's write-up and validation against arXiv:2610.08012 Table 2. `COVERAGE.md`: the per-prime table of codes.
- `docs/aanha_mc_guide/make_figures.py`: produced every figure and number in this document; run it to regenerate them.